UNIVERSIDAD VERACRUZANA
FACULTAD DE CONTADURIA
Y ADMINISTRACION
XALAPA, VERACRUZ

PROGRAMA EDUCATIVO
SISTEMAS COMPUTACIONALES ADMINISTRATIVOS

EXPERIENCIA EDUCATIVA
PROGRAMACION E IMPLEMENTACIÓN DE SISTEMAS

DOCENTE
GUZMAN COUTIÑO HECTOR

TRABAJO

DOCUMENTACIÓN TÉCNICA

COMBOWEB APP · EN COMBO

DOMINGUEZ LOPEZ JORGE ARTURO
DOMINGUEZ RUIZ OSCAR
HERNÁNDEZ VALERIO ABRIL MICHELLE
PARRA SALDAÑA AMAIRANI MARIEL

SECCIÓN
701

13 DE OCTUBRE DE 2026

ÍNDICE

1. INTRODUCCIÓN Y ALCANCE....3
Resumen....3
Tecnologías....3
2. ARQUITECTURA....5
Vista general....5
Decisiones de diseño....5
Flujo de una compra....6
3. ESTRUCTURA DEL PROYECTO....7
4. MODELO DE DATOS....8
5. REGLAS DE NEGOCIO....10
6. SEGURIDAD....11
Controles implementados....11
Limitaciones y riesgos conocidos....12
7. API....13
8. PRUEBAS....15
Estrategia....15
Suites y resultados....15
Defectos encontrados y corregidos durante las pruebas....15
Cómo ejecutar las pruebas....16
9. INSTALACIÓN PARA DESARROLLO....17
Requisitos....17
Pasos....17
Variables de entorno....17
10. DESPLIEGUE A PRODUCCIÓN....19
Preparación....19
Puesta en marcha....19
Checklist de salida a producción....19
11. OPERACIÓN Y MANTENIMIENTO....21
Respaldos y restauración....21
Actualizar a una nueva versión....21
Rutina de mantenimiento....21
Cómo agregar una ruta nueva....21
Qué hacer ante un incidente....22
12. TRAZABILIDAD DE LAS HISTORIAS DE USUARIO....23
13. TRABAJO FUTURO....24

1. INTRODUCCIÓN Y ALCANCE

Este documento describe cómo está construida **ComboWeb App**, la plataforma web de EN COMBO: su arquitectura, modelo de datos, reglas de negocio, seguridad, API, pruebas, instalación, despliegue y mantenimiento. Está dirigido a quien vaya a **operar, mantener o ampliar** el sistema. Para el uso diario del sistema, consulta el **Manual de usuario**.

La versión documentada es la **0.6.0**, resultado de los seis sprints del proyecto. Cubre las historias de usuario HU01 a HU12 del Product Backlog.

Resumen

Aspecto	Descripción
Tipo de sistema	Aplicación web con API REST, base de datos relacional y páginas servidas por el mismo servidor.
Roles	Cliente, cocina, almacén y administrador (control de acceso por rol en el servidor).
Módulos	Catálogo y pedidos, pagos, cocina y estatus, inventario, recetas y costeo, notificaciones por correo, dashboard y reportes, usuarios.
Tamaño aproximado	Unas 3,800 líneas de código (servidor, pantallas, scripts y esquema), más 6 suites de pruebas automatizadas.
Estado	Funcional y probado. Listo para desplegar tras completar el checklist de la sección 11.

Tecnologías

Componente	Versión	Uso
Node.js	≥ 18 (probado con 22)	Entorno de ejecución del servidor.
Express	4.19.2	Servidor HTTP, rutas y middleware.
PostgreSQL	≥ 14 (probado con 16)	Base de datos relacional con transacciones y bloqueos de filas.
pg	8.11.3	Conexión a PostgreSQL (consultas parametrizadas).
bcryptjs	2.4.3	Cifrado de contraseñas (bcrypt, costo 12).
jsonwebtoken	9.0.2	Sesiones firmadas (JWT, 8 horas).
helmet	7.1.0	Cabeceras de seguridad y política de contenido (CSP).
express-rate-limit	7.2.0	Límite de intentos en acceso y compras.
stripe	23.0.0	Pasarela de pago (Checkout alojado y webhook firmado).
nodemailer	10.1.0	Envío de correo por SMTP (también hay modo API HTTPS tipo Resend).

Componente	Versión	Uso
dotenv	16.4.5	Lectura de variables de entorno desde .env.
HTML, CSS y JavaScript	sin framework	Pantallas del sistema (sin paso de compilación).
jsdom, smtp-server, mailparser	solo pruebas	Navegador simulado y servidor SMTP local para las pruebas.

2. ARQUITECTURA

Vista general

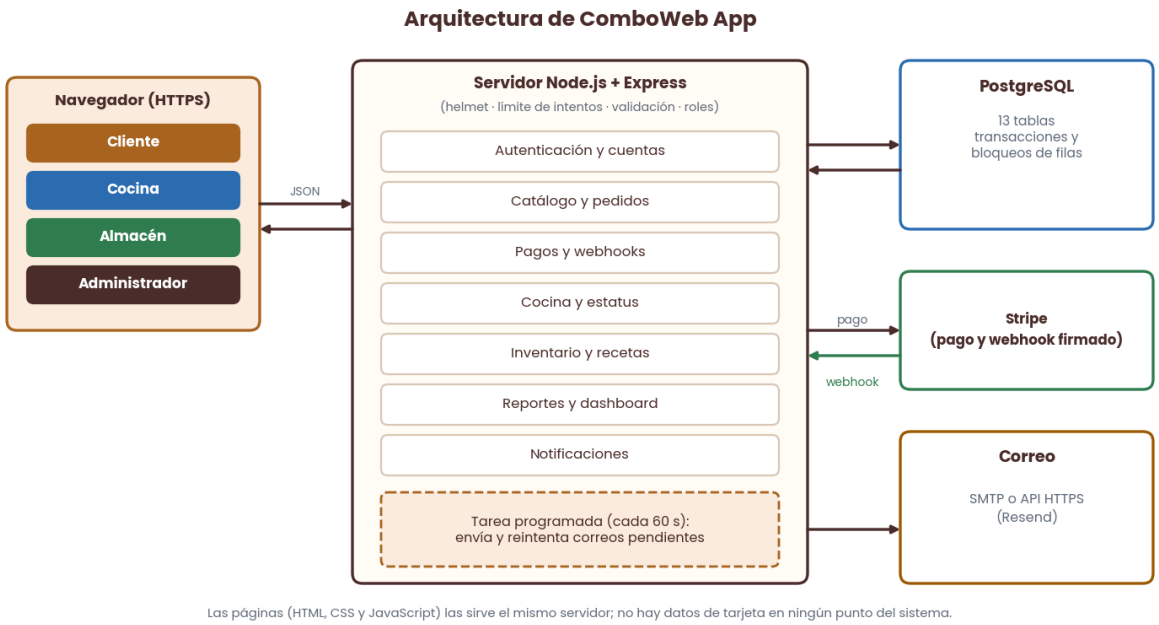


Figura 1. Arquitectura general de ComboWeb App.

El **navegador** carga las páginas desde el servidor y se comunica con él por una API JSON. El **servidor Node.js + Express** concentra toda la lógica (validaciones, permisos, transacciones); el navegador solo presenta información. **PostgreSQL** guarda los datos. El servidor habla con **Stripe** (cobro) y con el **servicio de correo**. Una **tarea programada** dentro del mismo proceso revisa cada minuto los correos pendientes y los reintentos.

Decisiones de diseño

Decisión	Razón
La lógica vive en el servidor, no en la pantalla.	Un usuario malintencionado puede alterar lo que corre en su navegador; todas las reglas (precios, permisos, inventario, horarios) se vuelven a validar en el servidor.
Los precios nunca viajan desde el cliente.	El servidor recalcula el total con los precios de la base de datos en cada cotización y cobro.
El pedido solo existe cuando el pago está confirmado.	Mientras el cliente paga se guarda un “checkout”; el pedido, el descuento de inventario, el pago y el correo se crean juntos en una sola transacción.
Correos con “bandeja de salida”.	El aviso se guarda en la misma transacción que el cambio de estatus; un fallo del correo nunca afecta al pedido y no se pierde ni se duplica.
Pantallas sin framework ni compilación.	Menos piezas que mantener; el código se construye con DOM seguro (textContent), lo que evita inyección de HTML.

Decisión	Razón
El rol y el estado de la cuenta se leen de la base de datos en cada petición.	Un cambio de rol o una baja aplica de inmediato, aunque la persona tenga una sesión abierta.

Flujo de una compra



Figura 2. Flujo de una compra, del armado del combo a la línea de tiempo del cliente.

Paso clave: al confirmarse el pago, la transacción **bloquea** los insumos involucrados y el horario, **vuelve a verificar** existencias y lugar, y recién entonces crea el pedido. Así, dos clientes que pagan al mismo tiempo por el último insumo o el último lugar **nunca** reciben ambos el pedido: al segundo se le marca el cobro "en revisión" para reembolsarlo.

3. ESTRUCTURA DEL PROYECTO

Ruta	Contenido
db/schema.sql	Esquema completo de la base de datos. Es **idempotente**: se puede ejecutar de nuevo sin borrar datos (agrega lo que falte).
src/server.js	Arranque, tarea de correos y apagado ordenado (SIGTERM).
src/app.js	Configuración de Express: seguridad, rutas, salud, manejo de errores.
src/config.js	Valida variables de entorno críticas al arrancar (el servidor se niega a iniciar con una configuración insegura).
src/middleware/auth.js	Verifica la sesión (token + versión) y el rol.
src/routes/*.js	Una archivo por área: auth, catalog, pedidos, pagos, webhooks, cocina, inventario, recetas, reportes, notificaciones, admin.
src/services/pedidos.js	Validación del combo, checkout y confirmación de pago (la transacción central).
src/services/notificaciones.js	Bandeja de salida de correos, reintentos y reclamo atómico.
src/payments/	Pasarelas: stripe.js (real) y simulado.js (solo desarrollo).
src/mail/	Correo: smtp.js, resend.js y consola.js (solo desarrollo).
public/	Pantallas: index.html, app.js (cliente), personal.js (shell, inventario y recetas), cocina.js, reportes.js, usuarios.js, pago-simulado.*.
scripts/	init-db, seed, crear-admin, verificar-despliegue y las suites de pruebas.

4. MODELO DE DATOS

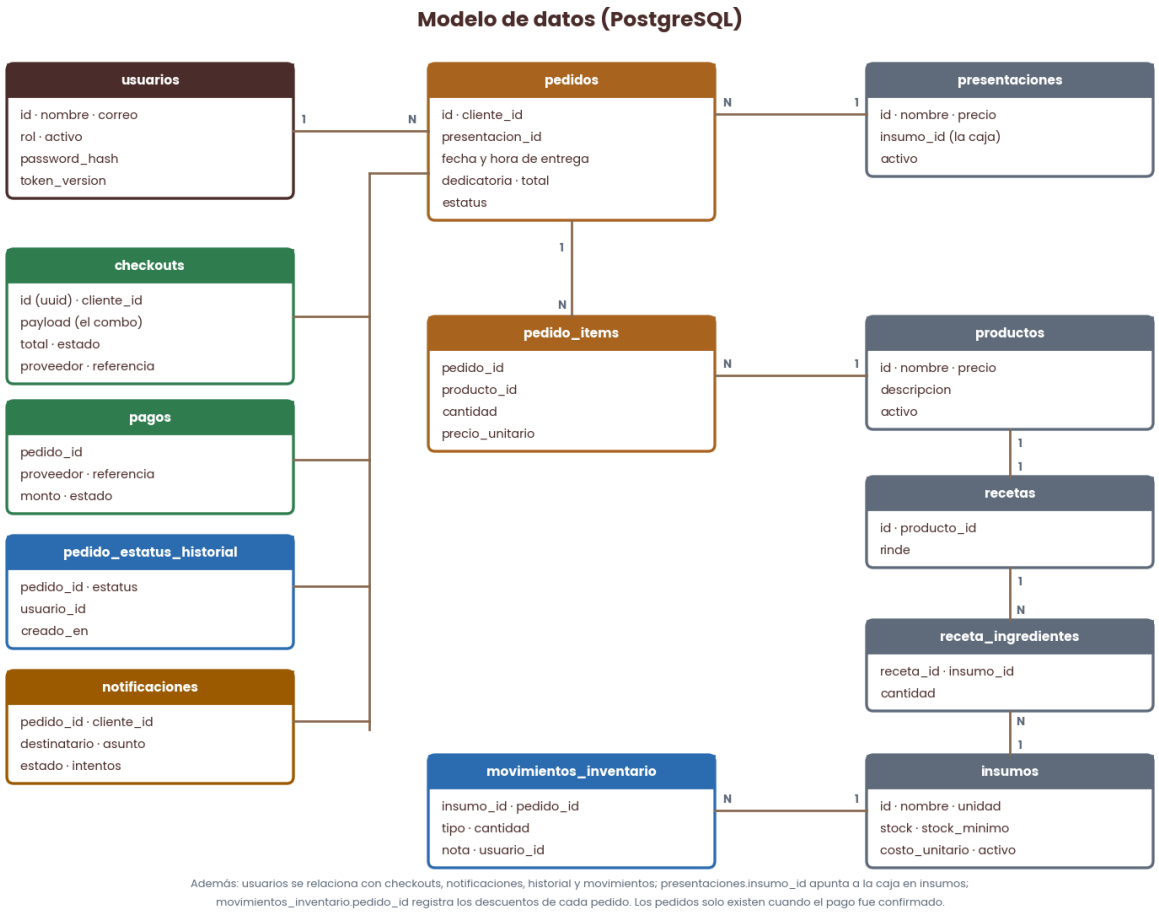


Figura 3. Modelo de datos: 13 tablas de PostgreSQL y sus relaciones.

Tabla	Propósito y reglas
usuarios	Cuentas de los cuatro roles. Correo único (sin importar mayúsculas), contraseña cifrada y **token_version**: al cambiar la contraseña sube y las sesiones anteriores dejan de valer.
productos / presentaciones	Catálogo. Un producto nace inactivo y solo se activa si tiene receta. Cada presentación está ligada al insumo “caja” que se descuenta.
insumos	Ingredientes y empaques: existencia, mínimo y costo unitario. Restricciones: existencia y mínimo ≥ 0 (la base de datos misma impide stock negativo).
recetas / receta_ingredientes	Qué lleva cada producto. **rinde** indica cuántas piezas salen; el consumo por pieza es cantidad ÷ rinde.
checkouts	Intento de compra mientras se paga (el combo en JSON, total, estado y referencia de la pasarela). Estados: creado, pagado, rechazado, expirado y **revision**.
pedidos / pedido_items	Pedidos confirmados y sus renglones (con el precio unitario del momento). Solo se crean tras un pago confirmado.

Tabla	Propósito y reglas
pagos	Cobros aprobados. Índice único (proveedor, referencia): el mismo cobro no se registra dos veces aunque el webhook se repita.
pedido_estatus_historial	Cada cambio de estatus con fecha, hora y quién lo hizo. Único por (pedido, estatus).
movimientos_inventario	Bitácora de entradas, salidas y ajustes; los descuentos por pedido quedan ligados al pedido.
notificaciones	Bandeja de salida de correos: estado, intentos, error y próximo intento. Único por (pedido, plantilla) para no duplicar avisos.

5. REGLAS DE NEGOCIO

Regla	Detalle
Cotización y validación	Productos activos con receta; 1–50 piezas por producto y máximo 10 productos distintos; dedicatoria de hasta 120 caracteres; fecha desde mañana y hasta 90 días (hora de México); hora entre 09:00 y 18:00.
Capacidad por horario	Máximo 2 pedidos por fecha y hora (configurable con SLOT_CAPACITY). Al llenarse se sugieren horarios alternativos.
Insumos suficientes	Se calcula el consumo de todo el combo (recetas ÷ rinde, redondeado hacia arriba a 4 decimales, más una caja) y se compara con la existencia, primero al cotizar y otra vez dentro de la transacción de pago.
Estatus del pedido	Pendiente → En preparación → Listo para entrega → Entregado. Solo hacia adelante y de uno en uno; repetir el mismo estatus no hace nada (doble clic seguro).
Inventario	El stock solo cambia con movimientos (entrada, salida, ajuste) y por los pedidos. Una salida nunca deja el stock negativo, ni con peticiones simultáneas.
Alertas de reorden	Existencia ≤ mínimo. Se calculan al momento; desaparecen solas al reponer.
Eliminar insumos	Si está en recetas o presentaciones: se rechaza y se indican cuáles. Con historial: se archiva. Sin historial: se borra.
Costeo	Costo del lote = Σ cantidad × costo unitario; costo por pieza = lote ÷ rinde; margen = (precio − costo por pieza) ÷ precio.
Reintentos de correo	A 1, 4, 9 y 16 minutos; al quinto fallo queda “fallida” y el administrador puede reintentarla.
Pagos en revisión	Si tras cobrar ya no hay insumos o lugar, o el monto/moneda no coinciden, no se crea pedido y el cobro queda en revisión para reembolso manual.
Último administrador	No se puede dar de baja ni quitar el rol al único administrador activo.

6. SEGURIDAD

Controles implementados

Control	Cómo se aplica	Cómo se verifica
Contraseñas	bcrypt (costo 12); 8–72 caracteres; nunca se devuelven en ninguna respuesta.	Pruebas de registro, cambio y respuestas del administrador.
Sesiones	JWT de 8 h con versión; se revalida contra la base en cada petición. Cambiar o restablecer la contraseña invalida las sesiones anteriores.	Tokens alterados, sin firma, caducados, de otra llave, de usuarios dados de baja o de otra versión → 401.
Control de acceso por rol	Middleware en cada ruta; el registro público solo crea clientes.	Barrido automático: 195 combinaciones ruta × rol. Una ruta nueva sin clasificar hace fallar la prueba.
Fuerza bruta	Límite de 20 intentos por 15 min en acceso, registro y cambio de contraseña; mensaje genérico de error y comparación a tiempo constante.	Servidor real: el intento 21 responde 429.
Inyección SQL	Todas las consultas son parametrizadas; los parámetros de agrupación se limitan a una lista permitida.	Intentos de inyección en login, filtros y agrupación.
Inyección de HTML (XSS)	Las pantallas escriben con textContent; los correos escapan el texto del cliente; CSP “self”.	Nombres y dedicatorias con código se muestran como texto en pantalla y correo.
Cabeceras	helmet: CSP, nosniff, anti-iframe, HSTS; sin x-powered-by; sin CORS.	Pruebas de cabeceras.
Pagos	Stripe Checkout alojado (el sistema no recibe tarjetas); webhook con firma verificada sobre el cuerpo crudo; idempotencia; comprobación de monto y moneda.	Firma falsa o alterada → 400; reenvío del evento sin duplicar; monto distinto → revisión.
Concurrencia	Transacciones, bloqueos de filas y candados por horario; bandeja de correos con reclamo atómico.	Pagos simultáneos por el último stock o lugar; 3 procesos enviando correos a la vez.
Validación de entradas	Tipos, rangos y longitudes en el servidor; cuerpo máximo de 100 KB (413); errores 4xx sin detalles internos.	Tipos inesperados, JSON roto y cuerpos enormes.
Archivos expuestos	Solo se sirve la carpeta public/.	/.env, /src, /db, /node_modules y rutas con “..” no se entregan.
Exportación CSV	Los textos que empiezan con = + - @ se neutralizan.	Prueba con un nombre que parece fórmula.

Control	Cómo se aplica	Cómo se verifica
Secretos y configuración	Solo en variables de entorno; el servidor se niega a arrancar en producción con pago o correo simulados o con un JWT_SECRET corto.	Pruebas de arranque con configuraciones inválidas.

Importante: Los cuatro avisos de seguridad que más importan al publicar: (1) cambiar o desactivar las **cuentas de prueba**; (2) usar un **JWT_SECRET** largo y aleatorio; (3) servir el sitio solo por **HTTPS**; (4) activar la **llave real** de la pasarela solo cuando todo esté probado. El comando `npm run verificar` revisa estos puntos.

Limitaciones y riesgos conocidos

- **Recuperación de contraseña:** no hay “olvidé mi contraseña” por correo; la restablece el administrador. Es un candidato claro para una siguiente versión.
- **Contraseñas temporales:** el sistema no obliga a cambiarlas en el primer ingreso (se pide por procedimiento).
- **Cierre de sesión:** el token vive en el navegador (sessionStorage) y no se revoca individualmente al “Cerrar sesión”; sí se invalida al cambiar la contraseña, dar de baja o expirar (8 h).
- **Límite de intentos en memoria:** vale para un solo proceso. Con varias instancias del servidor habría que moverlo a un almacén compartido (por ejemplo, Redis).
- **Reembolsos:** son manuales desde el panel de la pasarela (el sistema los detecta y avisa, pero no los ejecuta).

7. API

Todas las respuestas son JSON y, ante un error, incluyen un campo **error** con un mensaje en español. Las rutas protegidas requieren el encabezado **Authorization: Bearer <token>**. En total hay **46 rutas**; cada una está clasificada por acceso y verificada automáticamente.

Método	Ruta	Acceso	Descripción
GET	/api/salud	Público	Comprobación de salud (base de datos).
GET	/api/catalogo	Público	Productos y presentaciones activos.
POST	/api/auth/register	Público	Crea una cuenta de cliente.
POST	/api/auth/login	Público	Inicia sesión; devuelve el token.
GET	/api/auth/me	Sesión	Datos de la persona autenticada.
POST	/api/auth/cambiar-password	Sesión	Cambia la propia contraseña y cierra las demás sesiones.
GET	/api/pedidos/disponibilidad	Público	Horarios libres y ocupados de una fecha.
POST	/api/pedidos/cotizar	Público	Valida el combo y calcula el total (no guarda).
POST	/api/pedidos/checkout	Cliente	Valida, guarda el combo y devuelve el enlace de pago.
GET	/api/pedidos/checkout/:id	Cliente	Estado de un pago propio.
GET	/api/pedidos	Cliente	Mis pedidos con su línea de tiempo.
GET	/api/pedidos/:id	Sesión	Detalle y comprobante (dueño, cocina o administración).
POST	/api/pagos/webhook/stripe	Stripe (firma)	Confirmación de pago.
POST	/api/pagos/simulado/:id/aprobar	Cliente (desarrollo)	Pago simulado: aprobar. No existe en producción.
POST	/api/pagos/simulado/:id/rechazar	Cliente (desarrollo)	Pago simulado: rechazar.
GET	/api/cocina/pedidos	Cocina, admin	Tablero del día con fichas técnicas.
GET	/api/cocina/pedidos/:id	Cocina, admin	Ficha de un pedido.
PATCH	/api/cocina/pedidos/:id/estatus	Cocina, admin	Avanza el estatus (solo hacia adelante).
GET	/api/inventario	Almacén, admin	Lista de insumos (?inactivos=1 incluye archivados).
GET	/api/inventario/alertas	Almacén, admin	Insumos en reorden.
POST	/api/inventario	Almacén, admin	Crea un insumo.

Método	Ruta	Acceso	Descripción
PUT	/api/inventario/:id	Almacén, admin	Edita datos del insumo (no el stock).
POST	/api/inventario/:id/movimientos	Almacén, admin	Registra una entrada, salida o ajuste.
GET	/api/inventario/:id/movimientos	Almacén, admin	Historial de un insumo.
DELETE	/api/inventario/:id	Almacén, admin	Elimina o archiva un insumo.
GET	/api/recetas	Admin	Recetas con costos y márgenes.
GET	/api/recetas/:id	Admin	Una receta.
POST	/api/recetas	Admin	Crea la receta de un producto.
PUT	/api/recetas/:id	Admin	Reemplaza rinde e ingredientes.
DELETE	/api/recetas/:id	Admin	Borra una receta (producto inactivo).
GET	/api/reportes/dashboard	Admin	Resumen del día y avisos.
GET	/api/reportes/ventas	Admin	Ventas, costo y utilidad (&formato=csv).
GET	/api/reportes/estatus	Admin	Pedidos por estatus.
GET	/api/reportes/existencias	Admin	Existencias y su valor.
GET	/api/notificaciones	Admin	Avisos por correo y su estado.
POST	/api/notificaciones/:id/reintentar	Admin	Reintenta un aviso con error.
GET	/api/admin/productos	Admin	Todos los productos (con y sin receta).
POST	/api/admin/productos	Admin	Crea un producto (nace inactivo).
PUT	/api/admin/productos/:id	Admin	Edita nombre, descripción y precio.
PATCH	/api/admin/productos/:id/activo	Admin	Activa o desactiva (solo con receta).
GET	/api/admin/presentaciones	Admin	Presentaciones.
PUT	/api/admin/presentaciones/:id	Admin	Cambia precio o estado.
GET	/api/admin/usuarios	Admin	Lista de usuarios.
POST	/api/admin/usuarios	Admin	Da de alta personal.
PATCH	/api/admin/usuarios/:id	Admin	Cambia rol o estado (protege al último admin).
PATCH	/api/admin/usuarios/:id/password	Admin	Restablece una contraseña y cierra sus sesiones.

8. PRUEBAS

Estrategia

El proyecto se probó en cuatro niveles: **pruebas de API** (reglas, permisos y errores), **pruebas de concurrencia** (pagos y movimientos simultáneos), **pruebas de pantallas** en un navegador simulado, y **revisión visual con un navegador real** (Chrome sin interfaz) para detectar lo que las pruebas automáticas no ven. Todas las suites crean sus propios datos, los borran al terminar y restauran el inventario; deben usarse **solo en la base de datos de desarrollo**.

Suites y resultados

Suite	Comando	Pruebas	Qué cubre
Sprint 2	npm run test:smoke	33	Registro, acceso, roles, catálogo y administración; ataques básicos.
Sprint 3	npm run test:sprint3	70	Cotización y validaciones, checkout y pago simulado, webhook de Stripe con firma, pagos simultáneos por el último stock o lugar.
Sprint 4	npm run test:sprint4	92	Permisos, movimientos, alertas, eliminación segura, recetas, rinde, costeo y redondeo.
Sprint 5	npm run test:sprint5	96	Tablero y estatus, correos y reintentos (con un servidor SMTP real local y una API HTTPS simulada), dashboard y reportes con CSV.
Sprint 6	npm run test:sprint6	83	Seguridad de sesiones, barrido de 195 combinaciones ruta × rol, tokens manipulados, archivos expuestos, carga simultánea, scripts de despliegue y apagado ordenado.
Pantallas	npm run test:ui	85	Recorrido de los cuatro roles, reportes, usuarios, mi cuenta, seguimiento automático y ausencia de texto basura en pantalla.
Total		**459**	**Todas correctas (0 fallidas) con el código de esta versión.**

Defectos encontrados y corregidos durante las pruebas

La documentación incluye los defectos reales que las pruebas y la revisión visual encontraron, porque muestran por qué conviene mantener las suites:

Defecto	Corrección
Con varios movimientos de inventario simultáneos, el servidor se bloqueaba al agotar las conexiones a la base de datos (cada petición pedía una segunda conexión mientras retenía una).	Se reutiliza la conexión de la transacción. Hay pruebas de 40 movimientos y 14 compras simultáneas.

Defecto	Corrección
En el tablero de cocina, cambiar la fecha mientras se cargaba se ignoraba, y la respuesta en curso pisaba la fecha elegida.	Las cargas se encolan y se descarta la respuesta de una fecha ya superada.
Un cuerpo de más de 100 KB producía un error 500.	Ahora responde 413 sin ruido en el registro.
La palabra “null” aparecía suelta en pantalla (dashboard, inventario y otras tablas).	Guardia global y una prueba que busca “null”, “undefined” y “NaN” en pantalla.
Una lista de pedidos se rompía si algún pedido no traía productos.	La lista tolera datos incompletos.
El transporte de correo SMTP no detectaba un cambio de contraseña.	La llave del transporte incluye la contraseña.

Cómo ejecutar las pruebas

1. Configura el `.env` (con `SEED_PASSWORD`) y ejecuta `npm install` y `npm run db:init`.
2. Ejecuta `npm run db:seed` una vez (cuentas de prueba y datos de ejemplo).
3. Corre la suite que quieras. Excepto `test:smoke` (que necesita el servidor encendido y `BASE_URL`), ninguna requiere arrancar el servidor.

9. INSTALACIÓN PARA DESARROLLO

Requisitos

- Node.js 18 o superior y PostgreSQL 14 o superior.

Pasos

1. Crea la base de datos y un usuario propio en PostgreSQL (`CREATE USER combo_user WITH PASSWORD ...; CREATE DATABASE comboweb OWNER combo_user;`).
2. Descomprime el proyecto y ejecuta `npm install`.
3. Copia `.env.example` como `.env` y llena `DATABASE_URL`, `JWT_SECRET` (mínimo 32 caracteres aleatorios) y `SEED_PASSWORD`.
4. Ejecuta `npm run db:init` (crea las tablas) y `npm run db:seed` (datos de ejemplo y 3 cuentas de prueba).
5. Ejecuta `npm start` y abre http://localhost:3000.

Variables de entorno

Variable	Valor típico	Descripción
PORT	3000	Puerto del servidor.
DATABASE_URL	postgres://…	Cadena de conexión a PostgreSQL (obligatoria).
DATABASE_SSL	true	Usar conexión cifrada con la base (necesaria en bases alojadas en otro servidor).
JWT_SECRET	(48+ aleatorios)	Llave que firma las sesiones. Obligatoria, mínimo 32 caracteres.
NODE_ENV	production	En producción se prohíbe el pago y el correo simulados.
PUBLIC_URL	https://…	Dirección pública del sitio (enlaces de regreso del pago y de los correos).
TRUST_PROXY	1	Detrás de un proxy (Render, Railway…): hace que el límite de intentos use la IP real.
PAYMENT_PROVIDER	stripe	“simulado” (solo desarrollo) o “stripe”.
STRIPE_SECRET_KEY / STRIPE_WEBHOOK_SECRET	sk_live_… / whsec_…	Llaves de Stripe (de prueba: sk_test_…).
MAIL_PROVIDER	smtp	“consola” (solo desarrollo), “smtp” o “resend”.
MAIL_FROM	"EN COMBO <avisos@…>"	Remitente de los correos.
SMTP_HOST / PORT / USER / PASS		Datos del servidor SMTP (si MAIL_PROVIDER=smtp).

Variable	Valor típico	Descripción
RESEND_API_KEY	re_…	Llave de la API de correo por HTTPS (si MAIL_PROVIDER=resend).
SLOT_CAPACITY / MIN_DAYS_AHEAD	2 / 1	Pedidos máximos por horario y días mínimos de anticipación.
AUTH_RATE_LIMIT	20	Intentos de acceso por 15 minutos (se deja en blanco para el valor por defecto).
SEED_PASSWORD		Solo desarrollo: contraseña de las cuentas de prueba.

10. DESPLIEGUE A PRODUCCIÓN

Estos pasos son independientes del proveedor: aplican a un servicio gestionado (Render, Railway, Fly.io…) o a un servidor propio. El proyecto no depende de ninguno en particular; solo necesita **Node.js**, **PostgreSQL** y **HTTPS**.

Preparación

1. **Base de datos:** crea una base PostgreSQL gestionada (con respaldos automáticos si el proveedor los ofrece) y copia su cadena de conexión. Usa `DATABASE_SSL=true` si está fuera del servidor.
2. **Secretos:** genera un JWT_SECRET con `node -e "console.log(require('crypto').randomBytes(48).toString('hex'))"`.
3. **Pagos:** en Stripe, activa tu cuenta, copia la llave **real** (sk_live_…) y registra el endpoint del webhook `https://TU-DOMINIO/api/pagos/webhook/stripe` con los eventos `checkout.session.completed`, `checkout.session.async_payment_succeeded` y `checkout.session.expired`; copia su whsec_….
4. **Correo:** elige SMTP o Resend, verifica tu dominio de envío (para que los correos no caigan en no deseado) y obtén las credenciales. Algunos hostings gratuitos bloquean SMTP; en ese caso usa el modo `resend` (HTTPS).
5. **Variables:** configura en el proveedor todas las variables de la sección 9, con `NODE_ENV=production` y `TRUST_PROXY=1`.

Puesta en marcha

1. Comando de arranque: `npm start`. Comando de instalación: `npm install --omit=dev`.
2. Con la base de datos accesible, ejecuta una vez: `npm run db:init`. Opcional: `npm run db:seed` para cargar el catálogo de ejemplo (**en producción no crea cuentas de prueba**).
3. Crea al primer administrador: `ADMIN_EMAIL=… ADMIN_NAME="…" ADMIN_PASSWORD="…" npm run crear-admin`.
4. Ejecuta `npm run verificar` con las variables de producción. Debe terminar con “Listo para publicar” y sin ✖. Atiende los ⚠ (cuentas de prueba, llaves de prueba…).
5. Entra al sitio, inicia sesión como administrador, **crea las cuentas reales** del personal, ajusta el catálogo, los precios, los insumos con sus mínimos y las recetas.
6. Haz una **compra real de bajo monto** con tu propia tarjeta de punta a punta y confirma: pago, comprobante, correo, tablero, cambio de estatus y reembolso desde Stripe.
7. Configura el monitoreo con la ruta `/api/salud` (responde 200 si todo está bien y 503 si falla la base de datos).

Importante: Antes de cobrar de verdad, probar todo el flujo con las llaves de **prueba** de Stripe (tarjeta 4242 4242 4242 4242) y solo después cambiar a las llaves reales.

Checklist de salida a producción

Punto	Hecho	Cómo comprobarlo
`npm run verificar` sin ✖	☐	Salida del comando

Punto	Hecho	Cómo comprobarlo
Cuentas de prueba (@encombo.test) desactivadas o con contraseña cambiada	☐	Pantalla Usuarios
HTTPS activo y PUBLIC_URL con https://	☐	Candado del navegador
Webhook de Stripe registrado y probado	☐	Panel de Stripe → entregas del webhook
Correo de confirmación recibido en una compra real	☐	Bandeja de entrada y no deseado
Respaldo automático de la base de datos activo y una restauración probada	☐	Panel del proveedor
Personal capacitado y con su propia cuenta	☐	Plan de capacitación (Manual de usuario)
Aceptación firmada por la Product Owner	☐	Anexo A del Manual de usuario

11. OPERACIÓN Y MANTENIMIENTO

Respaldos y restauración

Los datos están solo en PostgreSQL. Usa los respaldos automáticos de tu proveedor y, además, una copia propia periódica:

Acción	Comando
Respaldo	`pg_dump "$DATABASE_URL" -Fc -f respaldo-AAAA-MM-DD.dump`
Restauración en una base vacía	`pg_restore -d "$DATABASE_URL" --no-owner respaldo-AAAA-MM-DD.dump`

Guarda los respaldos fuera del servidor y **prueba restaurar** al menos una vez antes de depender de ellos.

Actualizar a una nueva versión

1. Haz un respaldo.
2. Despliega el nuevo código y ejecuta `npm install --omit=dev`.
3. Ejecuta `npm run db:init` (es seguro repetirlo: solo agrega lo que falte).
4. Reinicia el servicio (el apagado ordenado termina las peticiones en curso) y comprueba `/api/salud`.

Rutina de mantenimiento

Frecuencia	Tarea
Diario	Revisar el dashboard: pagos en revisión y correos no enviados.
Semanal	Revisar alertas de reorden; comparar reportes de ventas y existencias.
Mensual	Probar una restauración de respaldo; ejecutar `npm audit` y actualizar dependencias con sus pruebas; revisar usuarios activos.
Cuando cambie el personal	Dar de baja las cuentas y revisar los roles.
Si se sospecha de una filtración	Cambiar JWT_SECRET (cierra **todas** las sesiones), las llaves de Stripe y del correo, y restablecer las contraseñas.

Cómo agregar una ruta nueva

Define la ruta con su `auth` y `roles(...)` y, **antes de terminar**, agrégala a la clasificación de `scripts/test-sprint6.js`. Esa prueba recorre todas las rutas del servidor con cada rol y falla si encuentra una ruta sin clasificar o con permisos distintos a los declarados: es la red de seguridad contra olvidos.

Qué hacer ante un incidente

Síntoma	Acción
El sitio no responde	Revisa `/api/salud`. Si devuelve 503, revisa la base de datos y su conexión (DATABASE_URL, SSL, límite de conexiones).
Pagos cobrados sin pedido	Dashboard → “pagos cobrados sin pedido” → reembolsar en Stripe usando la referencia → avisar al cliente.
No salen correos	Dashboard → avisos no enviados (muestra el error). Verifica el proveedor de correo y usa **Reintentar**.
Inventario que no cuadra	Revisa el historial del insumo (cada descuento queda ligado a su pedido) y registra un ajuste de conteo.
Una persona no puede entrar	Pantalla Usuarios: verifica que esté activa y restablece su contraseña.

12. TRAZABILIDAD DE LAS HISTORIAS DE USUARIO

HU	Historia	Implementación	Verificación
HU01	Armar y personalizar el combo	POST /pedidos/cotizar; pantalla Armar combo.	Suites Sprint 3 y Pantallas.
HU02	Dedicatoria opcional	Validación de 120 caracteres; ficha de cocina; comprobante.	Sprint 3; Pantallas (texto seguro).
HU03	Fecha y hora de entrega	GET /pedidos/disponibilidad; capacidad por horario.	Sprint 3 (horario ocupado y concurrencia).
HU04	Pago en línea	checkout + Stripe + webhook; transacción de confirmación; comprobante.	Sprint 3 (webhook, idempotencia, pagos simultáneos).
HU05	Seguimiento y notificaciones	Línea de tiempo; correos con bandeja y reintentos.	Sprint 5; Pantallas (actualización automática).
HU06	Tablero de cocina y estatus	/cocina/pedidos; estatus solo hacia adelante; historial.	Sprint 5; Pantallas.
HU07	Registrar y actualizar insumos	/inventario con movimientos e historial.	Sprint 4; Pantallas.
HU08	Descuento automático de inventario	Dentro de la transacción de confirmación del pago.	Sprint 3 y 4 (cantidades exactas y movimientos).
HU09	Alertas de reorden	/inventario/alertas; dashboard.	Sprint 4 y 5.
HU10	Recetas y costeo	/recetas con rinde, costo y margen.	Sprint 4; Pantallas.
HU11	Reportes	/reportes/*; CSV; tablas contables.	Sprint 5; Pantallas.
HU12	Catálogo, precios y usuarios	/admin/productos, /presentaciones y /usuarios; pantallas Catálogo y Usuarios.	Smoke (Sprint 2), Sprint 6 y Pantallas.

13. TRABAJO FUTURO

- Recuperación de contraseña por correo y cambio obligatorio de la contraseña temporal.
- Reembolsos desde el propio sistema (hoy son manuales en la pasarela).
- Costos históricos: guardar el costo de los insumos al momento de cada pedido para reportes contables exactos.
- Notificaciones por WhatsApp (el documento de tecnologías solo incluye correo).
- Cancelación o reprogramación de pedidos por el cliente, con sus reglas de reembolso.
- Límite de intentos y bandeja de correos compartidos para varias instancias del servidor.
- Pruebas con dispositivos móviles reales y con distintos navegadores.